Manejo de excepciones y validación de entradas (Bloque 7) / Qué hacer cuando algo falla: excepciones y validación

✓ **Hecho:** Ver

UNIDAD U1.3 · Python aplicado para integración con IA

Qué hacer cuando algo falla: excepciones y validación

Tu módulo ya está organizado en funciones claras, cada una con una responsabilidad definida. Pero como viste en la demostración del Bloque 5, y como reconociste en tu propio módulo al cierre del Bloque 6, todavía no maneja ningún error: si algo falla, se detiene por completo.

Esta lección explica el mecanismo con el que **Python** permite manejar esas fallas de forma controlada, y la idea de validación, que es su complemento. En la siguiente lección de este bloque vas a revisar la rúbrica completa de calidad de código, y en la que cierra la unidad vas a aplicar todo esto directamente a tu módulo.

Vale la pena que llegues a esa última lección con estos conceptos ya claros, porque ahí no vas a tener tiempo de aprenderlos y aplicarlos a la vez: **el objetivo de esta lección es justamente separar esas dos cosas**, para que cuando programes ya sepas qué estás construyendo y por qué.

⚠ Qué es exactamente una excepción

Cuando algo sale mal mientras un programa se está ejecutando (una conexión que no responde, un dato que no viene en el formato esperado), Python no simplemente falla en silencio: genera un objeto que representa ese problema específico, con información sobre qué ocurrió y en qué punto del código.



🚨 A eso se le llama **lanzar una excepción**. Si nada en tu código está preparado para recibir esa excepción, el programa se detiene ahí mismo, y lo único que se ve es un mensaje técnico: exactamente lo que le pasó al módulo de la demostración cuando se provocó el error de red a propósito.

Así se ve, en la práctica, ese mensaje técnico cuando nadie está preparado para recibirlo. Si el error ocurre dentro de una llamada real a un servicio (como la del Bloque 5), la consola muestra algo como esto:

```
Traceback (most recent call last):
  ...
requests.exceptions.ConnectionError: HTTPSConnectionPool(host='esta-direccion-no-existe.eje
```

Es útil pensar en la excepción como un aviso, no como un castigo. Python no está "rompiendo" tu programa a propósito: está reportando, de la forma más precisa que puede, que algo que tu código asumía como cierto dejó de serlo.

Qué hacer con ese aviso es una decisión que te corresponde a ti como quien escribe el código, no algo que Python decide por su cuenta.

🛡️ El mecanismo: intentar, y prever qué hacer si algo falla

La forma en que **Python** permite manejar esto de manera controlada es envolver la parte riesgosa del código (la que depende de algo externo, como una conexión de red) en una estructura que dice, en esencia: intenta ejecutar esto, y si algo sale mal en el camino, en vez de detener todo el programa, ejecuta esta otra parte que ya dejé preparada para ese caso.

Esa segunda parte es la que decide qué hacer con el problema: puede devolver un mensaje claro sobre qué falló, puede registrar el error para que quede evidencia de que ocurrió, o puede intentar una alternativa razonable. Lo importante es que el programa sigue con vida después del problema, en vez de terminar abruptamente.

> 📌 **Antes de ver el código**: los ejemplos de esta lección son ilustrativos, pensados para mostrar el mecanismo aislado, no fragmentos que debas copiar ya mismo en tu módulo. **"obtener_dato_de_prueba"** representa aquí, en general, tu función principal, la misma que ya construiste en el **Bloque 5** y organizaste en funciones en el Bloque 6 **("armar_peticion", "enviar_peticion"** y **"extraer_texto")**. Vas a aplicar este mismo mecanismo sobre esas funciones reales recién en la última lección de esta unidad.

En **Python**, ese mecanismo se ve así, en su forma más simple:

```
try:
     # el código que puede fallar
     resultado = obtener_dato_de_prueba(pregunta)
 except Exception as error:
     # qué hacer si algo sale mal
     print("No se pudo completar la operación:", error)
```

Ya viste esta misma estructura en el **Bloque 5**, usada para contener el error de red que se provocó a propósito. **"except Exception"** captura cualquier tipo de error; en la siguiente sección vas a ver por qué eso, aunque funciona, no siempre es la mejor opción.

🌐 Un ejemplo de lo que cambia, en un caso real

Imagina que tu módulo, ya terminado, forma parte de una aplicación que otras personas usan.

Sin manejo de errores

Sin **manejo de errores**, el día que el servicio externo tenga un problema momentáneo, toda la aplicación se cae para todos sus usuarios al mismo tiempo, por una causa que ni siquiera depende de tu código.

Con manejo de errores

Con **manejo de errores**, ese mismo problema momentáneo se convierte en un mensaje contenido: algo como "no se pudo obtener el resultado en este momento, intenta de nuevo en unos segundos". La aplicación sigue en pie, y quien la usa entiende qué pasó, en vez de encontrarse con una pantalla en blanco o un mensaje técnico que no significa nada para él.

⚖️ No todas las excepciones son iguales

Un error de conexión (el servicio no respondió) no es lo mismo que un error de formato (la respuesta llegó, pero no en la estructura **JSON** que tu código esperaba), y ninguno de los dos es lo mismo que un dato faltante (la respuesta llegó bien, pero no trae el campo específico que tu función busca).

Distinguir entre estos casos importa porque cada uno merece una respuesta distinta. Un error de conexión quizás justifica reintentar la petición una vez más; un dato faltante probablemente no, porque reintentar no va a hacer que ese dato aparezca. Tratar todos los errores por igual, con una sola respuesta genérica para cualquier cosa que salga mal, suele ocultar información valiosa sobre qué fue lo que realmente pasó.

Esta misma distinción es la razón por la que, más adelante, cuando escribas el código de manejo de errores de tu módulo, vas a preparar respuestas distintas para situaciones distintas, en vez de una única respuesta genérica del tipo "algo salió mal" para cualquier caso. Cuanto más específico el mensaje, más útil es para quien tiene que entender y corregir el problema.

Hay una razón adicional, más allá de la utilidad del mensaje: "except Exception" también atrapa errores de programación genuinos (un nombre mal escrito, un typo), no solo fallas externas como una conexión caída. En producción, eso significa que un error real en tu propio código puede quedar escondido detrás del mismo mensaje genérico que usas para una simple caída de red, hasta que ya es tarde para notarlo. Capturar excepciones específicas reduce ese riesgo: solo interceptas lo que realmente esperas que pueda fallar.

> **Python permite exactamente eso**: encadenar varios bloques "except", cada uno para un tipo de excepción distinto, en vez de uno solo genérico.

Aquí también **"obtener dato de prueba"** es un genérico ilustrativo: en tu propio módulo, este patrón de

encadenar "except" no va a vivir todo junto en un solo lugar, sino repartido según dónde ocurra cada falla, el error de conexión dentro de **"enviar_peticion"**, y el dato faltante dentro de **"extraer_texto"**, cada uno en la función que ya le corresponde desde el Bloque 6:

```
try:
    resultado = obtener_dato_de_prueba(pregunta)
except requests.exceptions.ConnectionError:
    print("No se pudo conectar con el servicio. Intenta de nuevo en unos segundos.")
except KeyError:
    print("La respuesta llegó, pero no trae el campo esperado.")
```

Los tipos de excepción de este ejemplo ("requests.exceptions.ConnectionError", "KeyError") son ilustrativos, para mostrar el mecanismo de encadenar varios "except"; cuáles usar exactamente en tu propio módulo depende de qué falla estés anticipando, y lo vas a definir cuando apliques esto en la última lección de esta unidad.

Validar antes de usar, no solo reaccionar después

⚠️ El **manejo de excepciones** reacciona a un problema una vez que ya ocurrió.

→

La **validación** es el complemento: revisar, antes de usar un dato, que tiene sentido usarlo.

Por ejemplo, confirmar que un identificador no llegue vacío antes de intentar armar una URL con él, en vez de dejar que el error aparezca recién cuando la petición ya falló.

> 🤝 **Las dos prácticas no compiten entre sí, se complementan**: la validación evita algunos problemas antes de que ocurran, y el manejo de excepciones se hace cargo de los que, aun así, van a pasar, porque dependen de algo fuera de tu control, como una conexión de red.

Una forma simple de decidir cuál usar en cada caso:





Si el problema depende de algo que tú controlas directamente (los datos que le pasas a tu propia función), **valídalo antes de usarlo.**

Si el problema depende de algo externo que puede fallar aunque todo de tu lado esté correcto (una conexión de red, un servicio de terceros), **prepárate para manejarlo con una excepción.**

Validar, en **código**, suele verse así: una comprobación simple al principio de la función, antes de que el resto del código se ejecute con un dato que no tiene sentido. Concretamente, en tu propio módulo, esta validación va al principio de **"armar_peticion"**, la función que arma la **URL** y el cuerpo de la petición: antes de armar nada, confirma que la pregunta recibida no venga vacía.

```
def armar_peticion(pregunta):
     if not pregunta:
        raise ValueError("La pregunta no puede estar vacía")
     # el resto de la función sigue igual
     ...
```

"raise" *es la forma en que tu propio código lanza una excepción a propósito, en vez de esperar a que Python la genere por su cuenta más adelante. Es la misma idea de las excepciones que ya viste, solo que ahora la disparas tú, de forma deliberada y temprana, en el momento más útil para diagnosticar el problema.*



🎯 Qué significa manejar bien un error, no solo evitar que el programa se detenga

Hay una forma de "manejar" errores que en realidad no ayuda: interceptar la excepción y no hacer nada con ella, dejando que el programa siga como si nada hubiera pasado. Eso evita que el programa se detenga, pero también esconde el problema, y esconder un problema casi siempre es peor que dejar que se vea.

Un buen manejo de errores informa con claridad qué pasó, en un lenguaje que tenga sentido para quien esté usando tu módulo, no solo para quien esté mirando el código en ese momento. Ese es, precisamente, uno de los criterios que la rúbrica de calidad de código usa para evaluar esta parte de tu trabajo, y es exactamente lo que revisa la siguiente lección de este bloque.



> **Buenas prácticas**: mensaje para el usuario, mensaje para quien debuggea.

No son el mismo mensaje.

➡️ A quien usa tu módulo le sirve algo simple y accionable, como "no se pudo completar la operación, intenta de nuevo en unos segundos", sin detalles técnicos que no puede interpretar.

➡️ A quien tenga que revisar el código después (tú mismo, en unas semanas) le sirve el detalle completo: el objeto de excepción entero, el mismo "error" que ya capturaste en el "except", porque ahí está la causa real y la línea exacta donde ocurrió.

Imprimir ambos juntos, como en los ejemplos de esta lección, funciona bien mientras trabajas solo; más

adelante, cuando tu módulo forme parte de algo más grande, vas a querer separarlos: uno hacia quien usa la aplicación, el otro hacia donde tú puedas revisarlo con calma.

Siguiente lección

Bloque 7 · Rúbrica de calidad de código

Con estas dos ideas ya claras (el **mecanismo de las excepciones**, y **la validación como su complemento**), estás listo para revisar, en la siguiente lección, cómo se traduce todo esto en criterios concretos de evaluación dentro de la **rúbrica de calidad de código**.

Última modificación: jueves, 24 de septiembre de 2026, 15:38